

SYSTEM PROGRAMMING AND FILE I/O USING C++

Report

Submitted By

Sanjay Pushparaj

Bachelor of Technology – Artificial Intelligence and Machine Learning
SNS College of Technology, Coimbatore

1. Introduction

System Programming is a branch of programming that focuses on developing software that interacts directly with the operating system and hardware resources. It includes tasks such as file management, process handling, command execution, and system utilities.

File Input/Output (File I/O) is one of the most important aspects of system programming because applications frequently need to store, retrieve, and process data from files. Understanding file handling is essential for building real-world software applications.

The objective of this assignment is to implement practical system programming utilities using C++. Three different programs were developed to demonstrate file operations, command execution, and text processing.

The utilities implemented are:

1. Log Management System
2. Simple Command Runner
3. File Statistics Analyzer

These applications demonstrate core concepts of system programming and provide practical experience with C++ file handling mechanisms.

2. Objectives

The primary objectives of this project are:

- To understand file input and output operations in C++.
- To implement practical utilities using file streams.
- To execute operating system commands from a C++ application.
- To process and analyze text files.
- To strengthen understanding of system programming concepts.
- To improve programming and problem-solving skills.

3. Development Environment

The project was developed using the following tools and technologies:

Component/Description

Programming Language

C++

Compiler

GNU G++

IDE

Visual Studio Code

Operating System

Windows 11

Version Control

GitHub

Compiler Version:

g++ (MinGW-W64) 15.2.0

The applications were compiled and executed using the Windows Command Prompt.

4. Project Structure

The project is organized into separate modules for better readability and maintainability.

Task3-SystemProgramming/

```
|
|—— LogManagement/
|   |—— logmanager.cpp
|
|—— CommandRunner/
|   |—— commandrunner.cpp
|
|—— FileAnalyzer/
|   |—— fileanalyzer.cpp
|
|—— screenshots/
|
|—— README.md
```

The screenshots folder contains program execution screenshots used for documentation purposes.

5. Log Management System

Overview

The Log Management System is a file-based utility that allows users to create log files, store messages, view log contents, and archive existing logs. This project demonstrates various file handling operations available in C++.

Features

- Create a new log file.
- Add log entries.
- Read and display existing logs.
- Archive logs into a separate file.
- Menu-driven interface.

File Operations Used

The following file stream classes were used:

- ifstream (Input File Stream)
- ofstream (Output File Stream)
- ios::app (Append Mode)

Working Procedure

1. Create a log file.
2. Enter log messages.
3. Store logs in a text file.
4. Display stored logs.
5. Archive logs when required.

Sample Output

===== LOG MANAGEMENT SYSTEM =====

1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit

Enter choice: 1

Log file created successfully.

Screenshot

Insert Log Management System screenshot here.

6. Simple Command Runner

Overview

The Simple Command Runner acts as a mini command shell that allows users to execute operating system commands directly from the application. The program uses the `system()` function provided by C++ to communicate with the operating system.

Features

- Execute system commands.
- Run multiple commands continuously.
- Exit when desired.
- User-friendly interface.

Example Commands

`dir`

`hostname`

`whoami`

`ipconfig`

Working Procedure

1. User enters a command.
2. Program sends the command to the operating system.
3. Operating system executes the command.
4. Output is displayed on the screen.

Benefits

- Demonstrates interaction with the operating system.
- Introduces process execution concepts.
- Useful for understanding shell functionality.

Sample Output

```
===== SIMPLE COMMAND RUNNER =====
```

Enter command:

`hostname`

DESKTOP-XXXXXX

Screenshot

Insert Command Runner screenshot here.

7. File Statistics Analyzer

Overview

The File Statistics Analyzer reads a text file and calculates various statistics related to the file contents.

The utility processes text files and generates useful information such as line count, word count, and character count.

Features

- Open a text file.
- Count lines.
- Count words.
- Count characters.
- Display results.

Working Procedure

1. User specifies a filename.
2. Program opens the file.
3. File contents are processed line by line.
4. Statistics are calculated.
5. Results are displayed.
- 6.

Sample Output

File Statistics

Lines : 15

Words : 120

Characters : 740

Screenshot

Insert File Analyzer screenshot here.

8. Compilation Commands

The following commands were used to compile the programs.

Log Management System

```
g++ LogManagement\logmanager.cpp -o LogManagement\logmanager.exe
```

Command Runner

```
g++ CommandRunner\commandrunner.cpp -o  
CommandRunner\commandrunner.exe
```

File Analyzer

```
g++ FileAnalyzer\fileanalyzer.cpp -o FileAnalyzer\fileanalyzer.exe
```

9. Execution Commands

Log Management System

LogManagement\logmanager.exe

Command Runner

CommandRunner\commandrunner.exe

File Analyzer

FileAnalyzer\fileanalyzer.exe

10. Concepts Demonstrated

The project demonstrates several important concepts related to system programming.

File Handling

- File Creation
- File Reading
- File Writing
- File Appending
- File Archiving

System Programming

- Operating System Interaction
- Command Execution
- Process Invocation

Text Processing

- Character Counting
- Word Counting
- Line Counting

User Interaction

- Menu Driven Applications
- Input Validation
- Console Based Interface

11. Learning Outcomes

The following skills were developed through this assignment:

- Understanding of File I/O Operations
- Practical Usage of C++ File Streams
- Command Execution Techniques
- Text Processing Algorithms
- System Utility Development
- Program Organization and Documentation
- Debugging and Testing

The project also improved understanding of how software interacts with operating system resources.

12. Conclusion

This project successfully implemented three practical utilities demonstrating System Programming and File I/O concepts in C++.

The Log Management System provided experience with file creation, reading, writing, and archiving. The Simple Command Runner demonstrated interaction with operating system commands. The File Statistics Analyzer showcased text processing and file analysis techniques.

The assignment strengthened knowledge of file handling, command execution, and system-level programming concepts while improving overall programming proficiency in C++. These concepts are essential for developing real-world software applications and form a strong foundation for advanced systems programming topics.

Output

```
File Edit Selection View Go Run ... LogManagement
PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS logmanager + - | X

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

D:\Projects\Internship\Task 3\LogManagement>g++ logmanager.cpp -o logmanager.exe

D:\Projects\Internship\Task 3\LogManagement>logmanager.exe

===== LOG MANAGEMENT SYSTEM =====
1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit
Enter choice: 1
Log file created successfully.

===== LOG MANAGEMENT SYSTEM =====
1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit
Enter choice: 2
Enter log message: Server Started
Log added successfully.

===== LOG MANAGEMENT SYSTEM =====
1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit
Enter choice: 3

----- LOG CONTENT -----
Server Started
```

```
File Edit Selection View Go Run ... LogManagement
PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS logmanager + - | X

===== LOG MANAGEMENT SYSTEM =====
1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit
Enter choice: 4
Logs archived successfully.

===== LOG MANAGEMENT SYSTEM =====
1. Create Log File
2. Add Log Entry
3. View Logs
4. Archive Logs
5. Exit
Enter choice: █
```